

Monitoramento Proativo de Nós HPC com Machine Learning

Pipeline de ML: manipulação de dados · modelos · métricas · integração com o scheduler

Fernando & Tiago · Orientador: Prof. João Paulo

Contexto atual de High Performance Cluster (HPC)/Cloud

- O crescimento de IA aumentou a dependência de clusters GPU em ambientes HPC
- Falhas em nós computacionais geram desperdício energético e perda de tempo computacional
- Skrzeczek (2024) destaca que abordagens puramente reativas se tornaram insuficientes

Impacto das falhas em sistemas HPC

- Falhas de CPU, memória, rede e armazenamento podem causar degradação em cascata
- Ambientes virtualizados sofrem com *resource contention* e interferência entre *workloads*
- Mohammed et al. (2019) ressaltam que a degradação de componentes afeta estabilidade e disponibilidade do cluster

HPC: Conjunto de computadores interligados em rede que operam como um único sistema para executar tarefas de alto desempenho

Evolução do estado da arte

- A literatura evoluiu de monitoramento tradicional para Machine Learning preditivo
- Cai et al. (2025) tratam falhas como processos graduais de degradação temporal
- O foco atual está em análise contínua de telemetria e previsão antecipada de falhas

Importância das métricas GPU

- Métricas GPU se tornaram relevantes em workloads modernos de IA e HPC
- Temperatura, potência e uso de memória refletem comportamento operacional do nó
- Skrzeczek (2024) mostra maior capacidade preditiva utilizando telemetria GPU integrada

Nó: Máquina individual dentro do cluster. Possui CPU, memória, disco e múltiplas GPUs dedicadas. É a unidade básica de alocação: cada job é atribuído a um ou mais nós

Objetivo do trabalho

- Hipótese principal: **Falhas em HPC surgem como degradação gradual detectável via telemetria, permitindo previsão com Machine Learning**
- O objetivo do trabalho é aplicar monitoramento proativo e manutenção preditiva em nós HPC com GPU
- A proposta acompanha tendências atuais de observabilidade inteligente, confiabilidade operacional e automação de infraestrutura

Vocabulário

- **Cluster HPC** — Conjunto de computadores interligados em rede que operam como um único sistema para executar tarefas de alto desempenho
- **Nó (Node)** — Máquina individual dentro do cluster. Possui CPU, memória, disco e múltiplas GPUs dedicadas. É a unidade básica de alocação: cada job é atribuído a um ou mais nós
- **Job** — Unidade de trabalho submetida à um nó. Tem timestamps de início e fim, e termina com um estado final registrado pelo Slurm: COMPLETED, FAILED, TIMEOUT, OUT_OF_MEMORY ou NODE_FAIL. É a principal fonte de rótulos neste trabalho
- **Slurm** — Gerenciador de recursos e scheduler do cluster. Decide qual nó executa cada job, controla filas de espera, registra o histórico de execuções
- **Snapshot** — Leitura pontual de todas as métricas de um nó em um instante específico

Etapas do tratamento de dados

- 1. Carregamento — dataset SURF Lisa completo, 14 colunas selecionadas
- 2. Rotulação — *label*=1 nas 2h antes do fim de *jobs* de falha, *label*=0 para todos os demais
- 3. *Split* temporal — corte por *end_date* do *job*; 80% treino / 20% teste
- 4. Janela deslizante — 40 snapshots por *job*; min/max/mean/std
- 5. Treino e avaliação — Random Forest ou HGB

Dataset

- Proveniente do data center acadêmico SURF Lisa - tabela *prom_slurm_joined*
- 100 Colunas, 13,5 M de linhas
- Distribuição das colunas da tabela:

Categoria	Quantidade	Exemplos
Métricas de nó (CPU, memória, rede, disco, sistema)	80	node_memory_Active_bytes, node_netstat_Tcp_InErrs, node_forks_total, node_load15...
Métricas de GPU (NVIDIA)	10	nvidia_gpu_temperature_celsius_mean, nvidia_gpu_memory_used_bytes_sum...
Metadados do job (Slurm)	10	slurm_id, state, submit_date, start_date, end_date, slurm_nodes, gpu_node...

Features selecionadas

- As 8 features foram selecionadas com base em Skrzeczek (2024), que identificaram estas métricas como as mais informativas para predição de falhas em nós HPC com GPU
- Métricas de nó:
 - node_memory_Active_bytes — pressão de memória; antecede falhas de OOM
 - node_disk_written_bytes_total_sum — atividade de disco; spikes podem indicar swap excessivo
 - node_netstat_Tcp_InErrs — erros TCP; anomalias de rede correlacionam com NODE_FAIL
 - node_forks_total — criação de processos; explosão de forks indica runaway processes
- Métricas de GPU:
 - nvidia_gpu_temperature_celsius_mean — overheating é causa direta de falha de hardware
 - nvidia_gpu_power_usage_milliwatts_mean — anomalias indicam comportamento fora do envelope normal
 - nvidia_gpu_memory_used_bytes_sum — esgotamento causa OOM_KILL
 - nvidia_gpu_fanspeed_percent_mean — resposta ao overheating; correlaciona com temperatura
- Metadados:
 - slurm_id, end_date, timestamp, state, gpu_node, node

Rotulação temporal por snapshot

- A coluna *state* registra o desfecho final do job
 - *state* de *jobs* de falha: FAILED, NODE_FAIL, TIMEOUT, OUT_OF_MEMORY
 - *state* de *jobs* de sucesso: COMPLETED
- Regra de rotulação:
 - *label* = 1 será atribuído aos *snapshots* nas 2h antes do *end_time* de *jobs* de falha
 - *label* = 0 será atribuído aos *snapshots* de *jobs* de sucesso e de *jobs* de falha antes de 2h de seus términos
- 2h é um período validado por Cai et al. (2025)
 - Entre 1h e 5h, captura-se degradação gradual (*overheating*, *memory pressure*) sem rotular como pré-falha momentos saudáveis do início do *job*

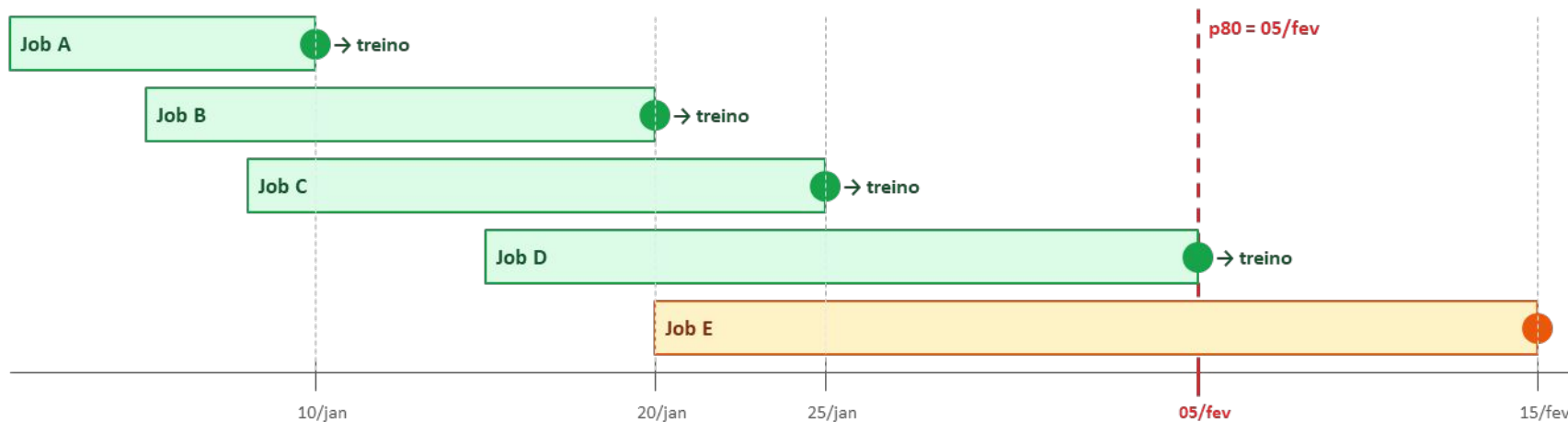
Rotulação temporal por snapshot

- Exemplo: *snapshots* de *job* aleatório (slurm_id = 1603441, estado = TIMEOUT)
- 2h = 7200s, 7200s/15s (tempo de coleta de cada *snapshot*) = **480 snapshots** antes de *end_time* recebem *label* = 1

snapshot	gpu_temp	gpu_power	gpu_mem	gpu_fan	mem_active	disk_wr	tcp_err	forks	label
t ₁ (start_time)	58	165	7.8	40	11.2	320	2	980	0
t ₂	59	168	7.9	41	11.3	325	2	1010	0
...	snapshots normais								0
t ₁₄₀	68	210	9.5	58	13.1	620	5	1800	1
t ₁₄₁	69	215	9.7	59	13.3	640	5	1850	1
...	zona de 2h pré-falha								1
t ₆₂₀ (end_time)	78	245	10.5	72	14.1	890	7	2300	1

Split treino/teste

- Para garantir que todos os *snapshots* fiquem em um lado do corte, a coluna *end_date* pode ser usada
 - *Jobs* com *end_date* antes do percentil 80 → treino
 - *Jobs* com *end_date* no percentil 80–100 → teste
- Exemplo com 5 *jobs*:



Janela deslizante para min/max/mean/std

- A janela deslizante percorre os snapshots de cada job (slurm_id) em ordem cronológica, agrupando 40 snapshots consecutivos (15s por snapshot = 10 minutos) por vez
- A cada passo, a janela avança 1 snapshot, gerando uma nova amostra (overlap de 39 snapshots entre janelas consecutivas). Label da janela é dado pela maioria de 0s ou 1s

janela	snapshots	label
W_1	$t_1 \rightarrow t_{40}$	0
W_2	$t_2 \rightarrow t_{41}$	0
...	...	0
W_{180}	$t_{180} \rightarrow t_{220}$	1
W_{181}	$t_{181} \rightarrow t_{221}$	1
...	...	1
W_{581}	$t_{581} \rightarrow t_{620}$	1

Janela deslizante para min/max/mean/std

- Cada janela produz 1 vetor de 32 valores

feature	min	max	mean	std
gpu_temp	68	78	73.2	3.1
gpu_power	210	245	228.5	11.4
gpu_mem	9.5	10.5	10.0	0.3
gpu_fan	58	72	65.3	4.5
mem_active	13.1	14.1	13.6	0.3
disk_wr	620	890	755	87.2
tcp_err	5	7	5.8	0.7
forks	1800	2300	2050	162

Janela deslizante para min/max/mean/std

- O *Input* final para o modelo são todas as janelas produzidas com seus labels

amostra	gpu_temp_min	gpu_temp_max	gpu_temp_mean	gpu_temp_std	... (×8)	forks_std	y
W_1	55	60	57.3	1.5	...	45	0
W_2	55	61	57.5	1.6	...	48	0
...	...					...	
W_{581}	68	78	73.2	3.1	...	162	1

Random Forest e HistGradientBoosting

Random Forest

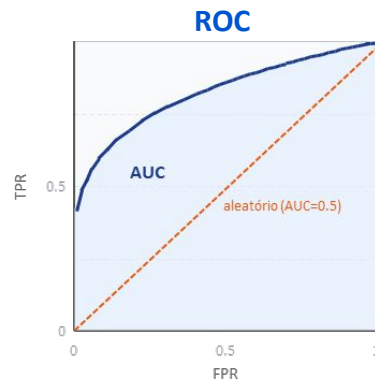
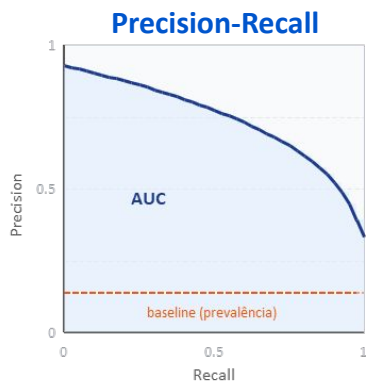
- Ensemble de árvores com *bootstrap* e seleção aleatória de *features* — robusto a ruído e *outliers*, esperados em métricas de sistema
 - Entrega importância de *features* nativamente: identifica quais métricas de hardware são mais informativas
 - Skrzeczek (2024): RF atingiu acurácia de 0,95 e ROC-AUC de 0,98 no mesmo *dataset* SURF Lisa, com as mesmas *features*

HistGradientBoosting

- *Gradient boosting* em histogramas: cada árvore corrige os erros da anterior — captura padrões não-lineares sutis
 - Tende a ser mais preciso na classe minoritária: o *boosting* dá mais peso aos exemplos difíceis de classificar
 - Skrzeczek (2024): HGB empatou com RF (0,95 de acurácia), superando AdaBoost, KNN, Decision Tree e Logistic Regression

PR-AUC e ROC-AUC

- PR-AUC (Precision-Recall Area Under the Curve)
 - Área sob a curva Precision-Recall. Penaliza modelos que ignoram a classe minoritária
 - Usada por Cai et al. (2025) como métrica principal pelo mesmo motivo: dataset desbalanceado
- ROC-AUC (Receiver Operating Characteristic Area Under the Curve)
 - Skrzeczek (2024) reportou ROC-AUC de 0,98 com RF no dataset



Precision: dos exemplos classificados como positivos pelo modelo, qual fração realmente era positiva;
Recall: dos exemplos que realmente eram positivos, qual fração o modelo conseguiu detectar.

Arquitetura da simulação

- Inspirada em Behera et al. (2020), com a arquitetura: preditor → decisor → ação proativa
- *Daemons mock* coletam métricas de CSVs sintéticos a cada ~15s e gravam em um banco de dados
- A partir do 40º *snapshot*, a janela desliza a cada nova coleta: agrega min/max/mean/std → vetor de 32 colunas → modelo retorna P(falha)
- *Slurm prolog* consulta o *score* mais recente antes de alocar um *job*; se $P(\text{falha}) > \text{threshold}$, o nó é drenado automaticamente
- Objetivo: validar a integração *end-to-end* da arquitetura

Daemons mock: Processo em segundo plano que simula a coleta de métricas de um nó real;

Slurm prolog: Script executado automaticamente pelo Slurm antes de cada job ser alocado a um nó.

Resumo

Dados | 100% dos nós GPU · CANCELLED excluído · 11,1M snapshots · 2,22 GB RAM

Features | 8 métricas (nó + GPU) baseadas em Skrzeczek 2024 e Cai et al. 2025 · 0% NaN

Rotulação | label=1 nas 2h antes do fim de jobs de falha · label=0 para todos os demais

Split | Corte por end_date do job · 80/20

Janela | 40 snapshots por slurm_id · min/max/mean/std · 32 features

Modelos | Random Forest + HGB · class_weight='balanced' · ambos 0,95 acurácia (Skrzeczek 2024)

Métricas | PR-AUC · ROC-AUC

Simulação | Daemon → SQLite → RF → prolog mock → drain + requeue

Cronograma

Junho | Refinamento do modelo: ajuste de features, hiperparâmetros e validação dos algoritmos.

Julho | Simulação: implementação e execução inicial das simulações.

Agosto | Simulação: testes adicionais, coleta de métricas e integração com scheduler.

Setembro | Análise de Resultados: avaliação de desempenho, comparação de métricas e interpretação dos resultados.

Outubro | Escrita do Artigo: documentação da metodologia, resultados e conclusão do trabalho.

REFERÊNCIAS BIBLIOGRÁFICAS

- CAI, Shengnan; NIE, Shuxin; CHEN, Zhehui; GULALKARI, Nupur; VANICA, George; JAIN, Chetna; SANKARAN, Sethuraman. Forecasting machine degradation of GPU Clusters. In: **NeurIPS 2025 Workshop on ML for Systems (MLForSys)**, 2025. Disponível em: <https://openreview.net/forum?id=18LJsdjevt>. Acesso em: mai. 2026
- MOHAMMED, Bashir; AWAN, Irfan; UGAIL, Hassan; YOUNAS, Muhammad. Failure prediction using machine learning in a virtualised HPC system and application. **Cluster Computing**, v. 22, p. 471–485, 2019. DOI: <https://doi.org/10.1007/s10586-019-02917-1>.
- SKRZECZEK, Ignacy. **Towards a more Robust and Efficient Failure Prediction in Cloud Systems**. 2024. Dissertação (Mestrado) – Graduate School of Natural Sciences, Universiteit Utrecht, Utrecht, 2024. Orientadores: Dr. Nishant Saurabh; Dr. A.A.A. (Hakim) Qahtan. Supervisor diário: Diogo Landau, MSc. Disponível em:
- BEHERA, Swapneel; WAN, Lipeng; MUELLER, Frank; WOLF, Matthew; KLASKY, Scott. **Orchestrating Fault Prediction with Live Migration and Checkpointing**. In: Proceedings of the 29th International Symposium on High-Performance Parallel and Distributed Computing (HPDC '20), p. 175–176, 2020. DOI: <https://doi.org/10.1145/3369583.3392672>.